



Vizuara AI Labs Presents

Machine Learning in Production - Lecture 1 Notes

Have you ever wondered whether your Kaggle project would actually work in the real world?

Sure, you might have achieved 100% accuracy on a Cats vs. Dogs classification task, but does that guarantee your model can accurately classify any image outside that dataset?

Now imagine scaling this up.

What if you're building a vehicle detection system for a government surveillance project?

Would running a Google Colab notebook be enough? **Probably not.**



At that stage, you need to think beyond the model:

Have you spoken with stakeholders?

Did you involve business developers or domain experts?

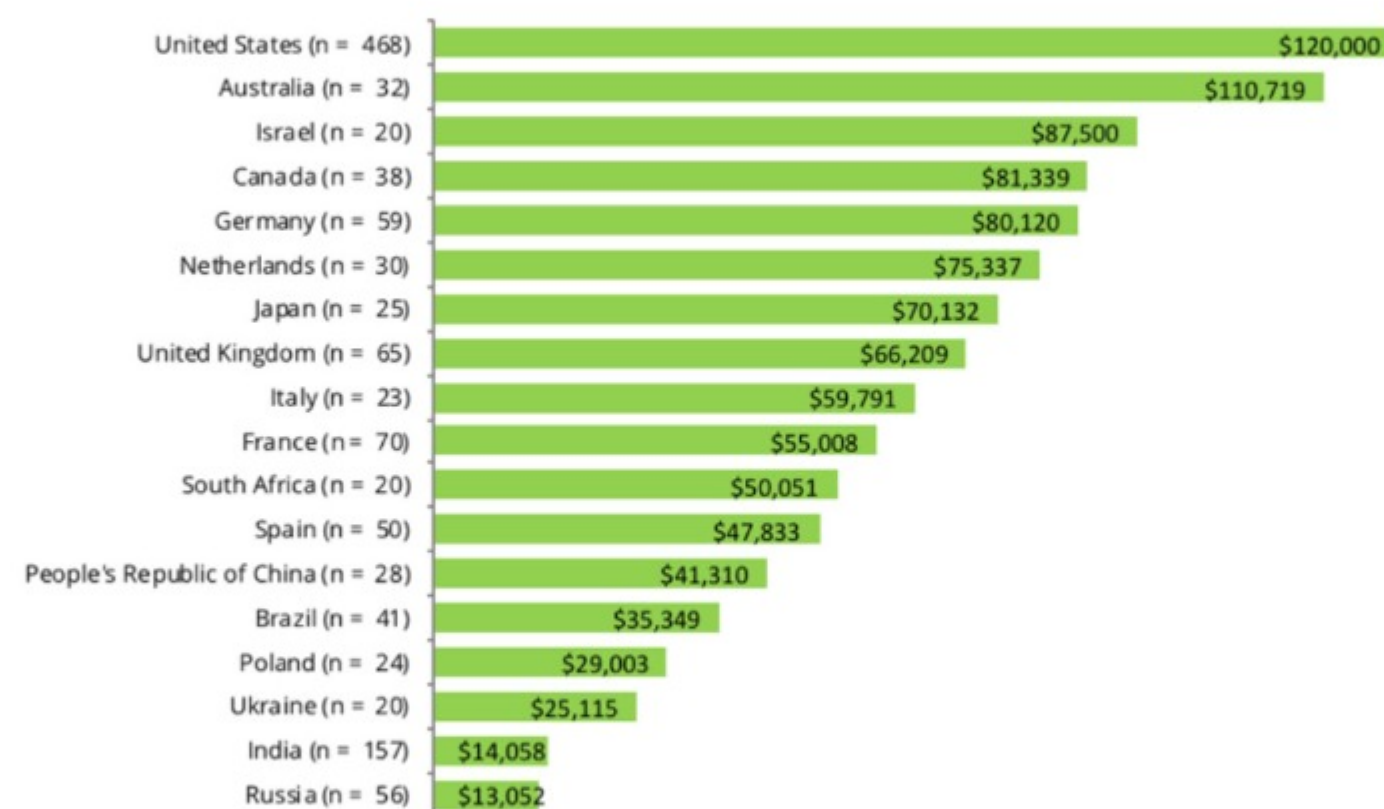
Are you thinking about scalability, deployment, and long-term maintenance?

This is where machine learning engineering meets real-world complexity and Machine Learning in Production Comes into

Machine learning (ML) is exciting. It's fun, challenging, creative, and intellectually stimulating.

Working in ML is tough, it involves mastering many tools, juggling data engineering and advanced analytics, and collaborating with experts across teams to solve problems and explain results.

Median Annual Compensation for Data Scientists and Machine Learning Engineers for Specific Countries



All values are in US Dollars. Conversion rates from mid 2017 (when the data were collected) were used for conversion.
Data are from the Kaggle 2017 The State of Data Science and Machine Learning study. You can learn more about the study and download the data here: <https://www.kaggle.com/surveys/2017>.
Countries are ranked by median annual salary. Only countries with ample sample size (n >= 20) are presented.



Copyright 2018 Business Over Broadway



Vizuara AI Labs

(1) Why ML Engineering

The aim of ML engineering is not to iterate through the lists of skills just mentioned and require that a data scientist (DS) master each of them.

The "Just Enough" Rule for ML Engineers

ML Engineers don't need to master everything, but they do need just enough of the right things:

1) **Software Dev**: Write clean, modular code and basic unit tests.

✗ No need to dive into advanced async systems or broker setups.

2) **Data Engineering**: Build and schedule ETL jobs for features.

✗ Skip building massive, real-time streaming systems.

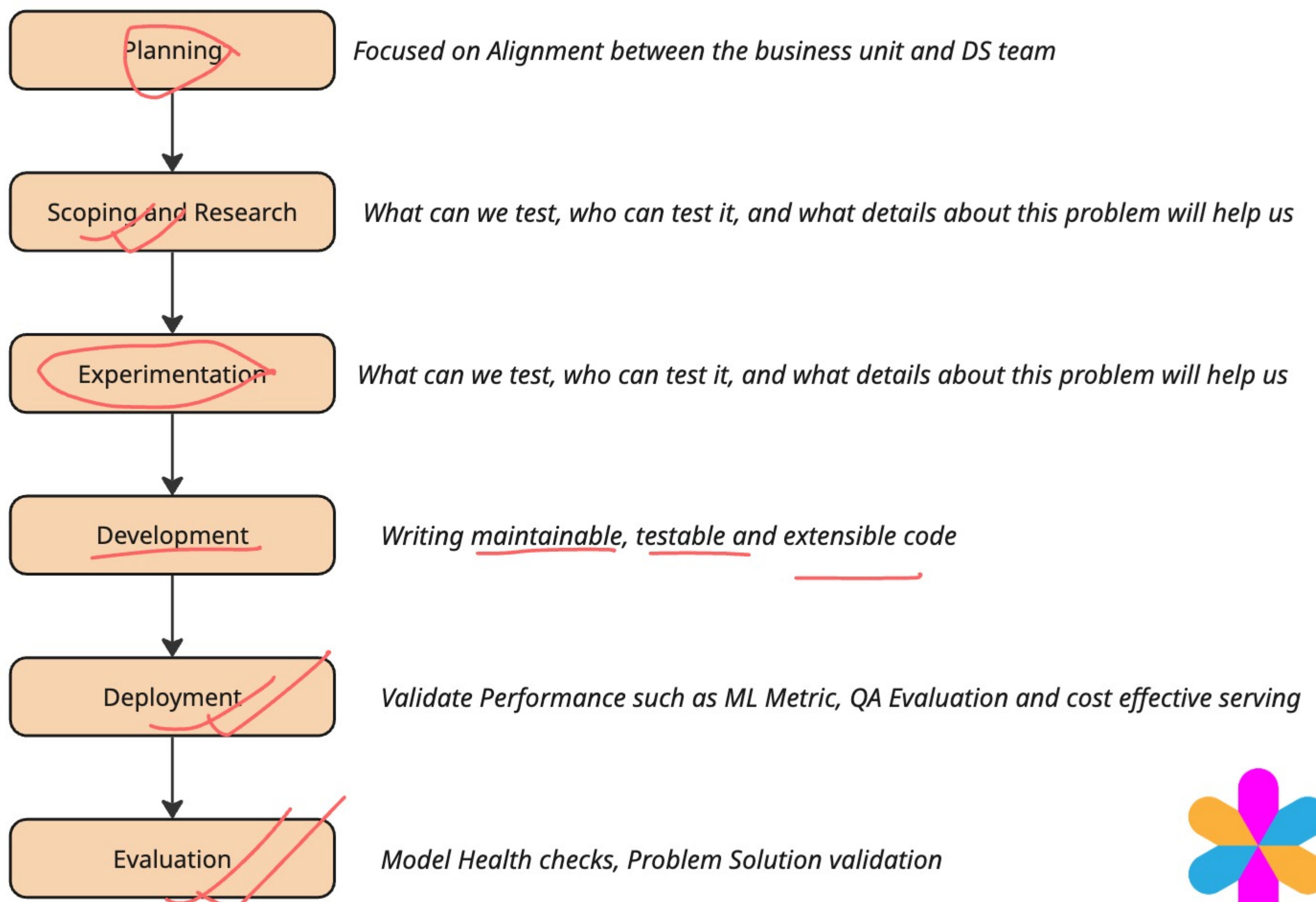
3) **Visualization**: Create simple, clear plots to explain model behavior.

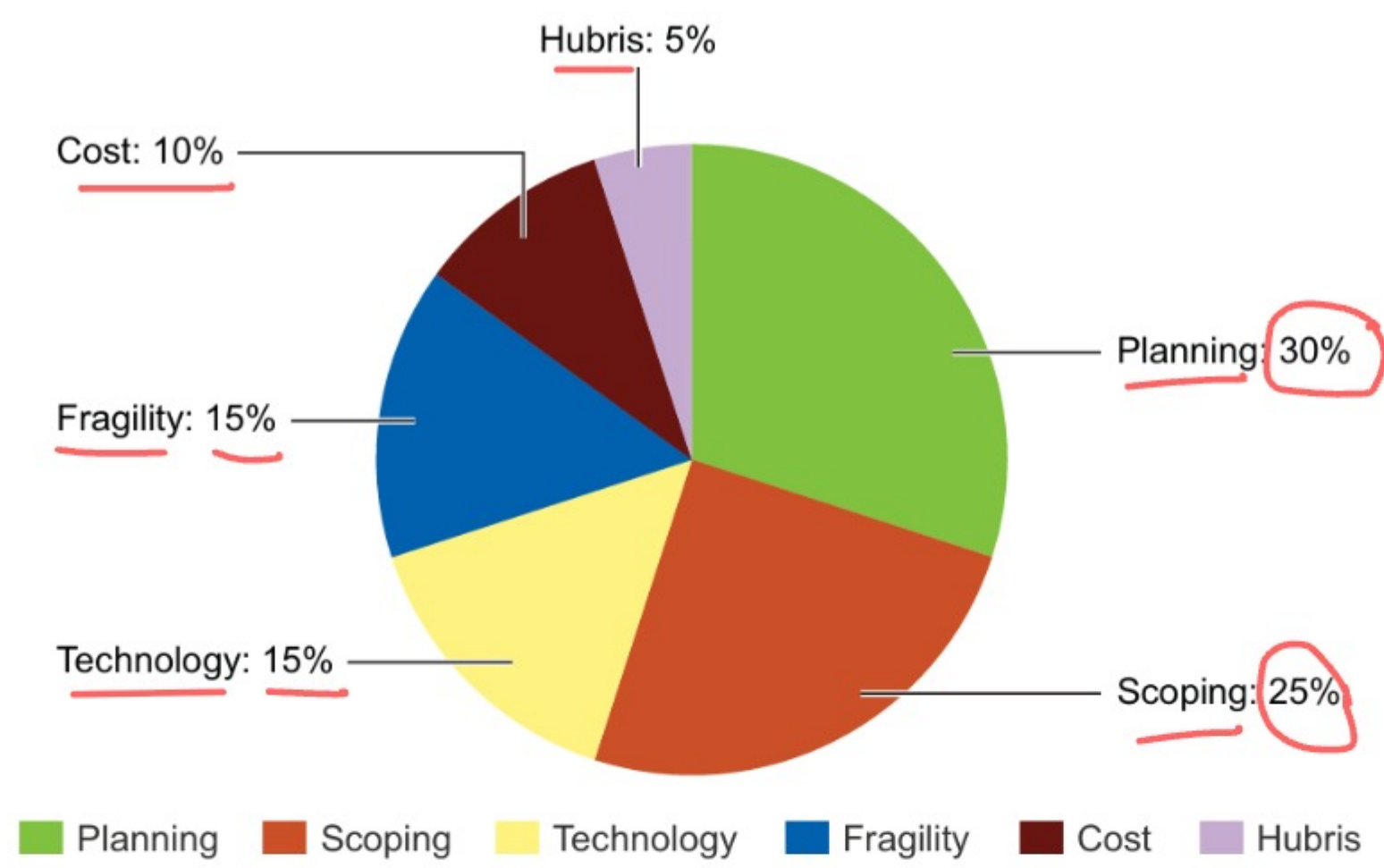
✗ No need for fancy, interactive UX-heavy dashboards.

4) **Project Management**: Define, scope, and steer ML projects with focus.

✗ No PMP certification required, clarity beats credentials here.

1.1 Major elements of ML project work





(Source:- *Machine Learning Engineering in Action*, Ben Wilson)

ML projects are often harder because, unlike typical software projects, they usually start with vague goals and rely on tools that are still evolving across the industry.

Just as software engineers decades ago refined their processes from large-scale waterfall implementations to a more flexible and productive Agile process, ML engineering seeks to define a new set of practices and tools that will optimize the wholly unique realm of software development for data scientists.

(2)The core tenets of ML engineering

2.1) Planning

Nothing is more demoralizing than building an ML solution that solves the wrong problem.

(Source:- *Machine Learning Engineering in Action*, Ben Wilson)

Imagine this:



- No clear goal
- No data
- No details



So you start thinking: -

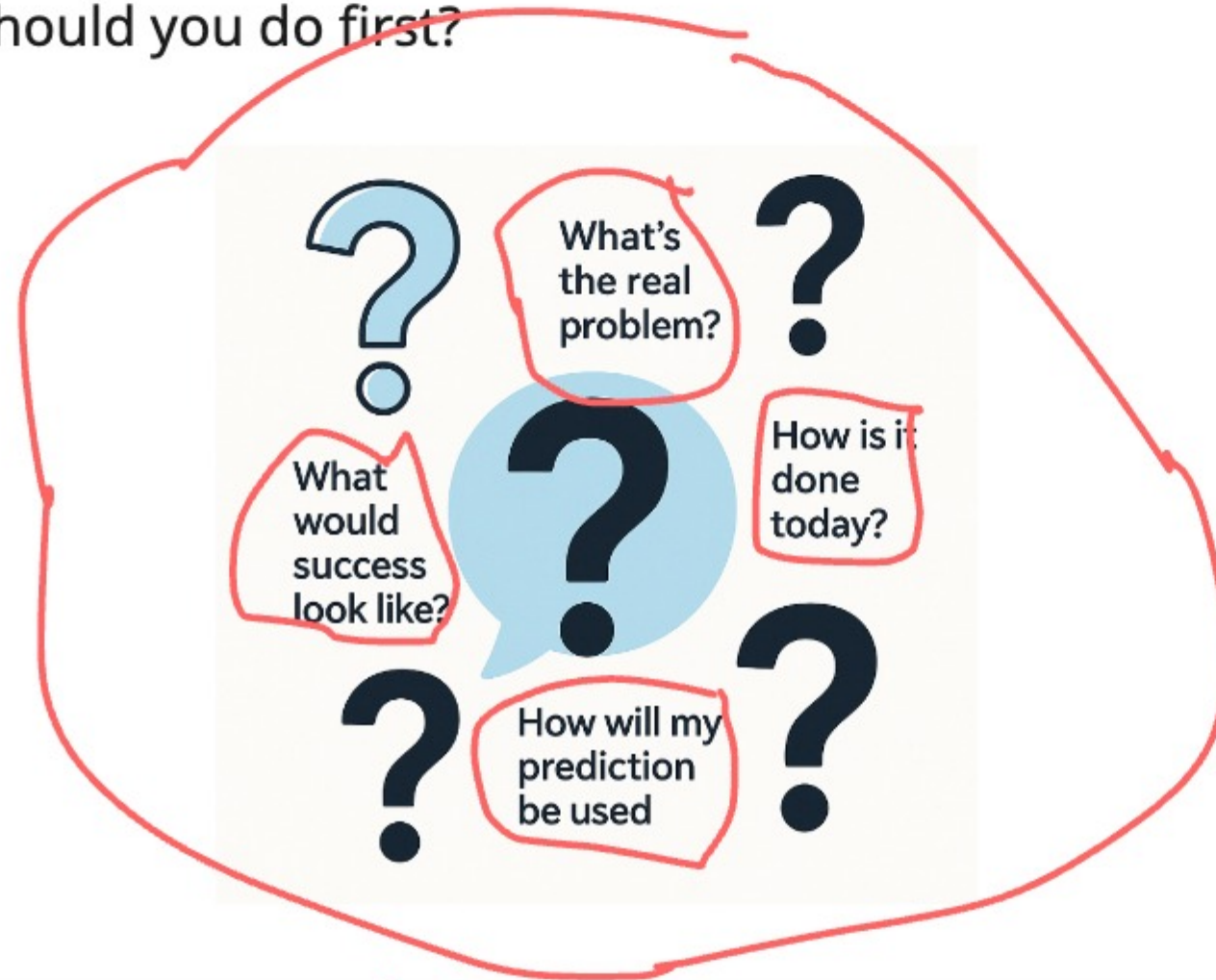
- 1) Should I build a smart subject line generator using NLP?
- 2) Should I recommend products based on user behavior?
- 3) Should I personalize email content? All cool ideas, right?

But here's the twist: None of that is what they actually need. All they really want is this: "Tell us the best time to send emails so people are more likely to open them." That's it.

Moral of the Story: Talk Before You Build Without asking the right questions early on, you risk building a super-complex system no one asked for. Worse?

You'll waste months, lose money, and probably kill the project.

So what should you do first?



ML success isn't just about models. It's about understanding people and problems first. A little planning can save a lot of pain and maybe even your job!

2.2) Scoping and Research

If you switch your approach halfway through development, you'll face a hard conversation with the business to explain that the project's delays are due to you not doing your homework.

(Source:- *Machine Learning Engineering in Action*, Ben Wilson)

When business leaders ask about an ML project, they care about just two things:

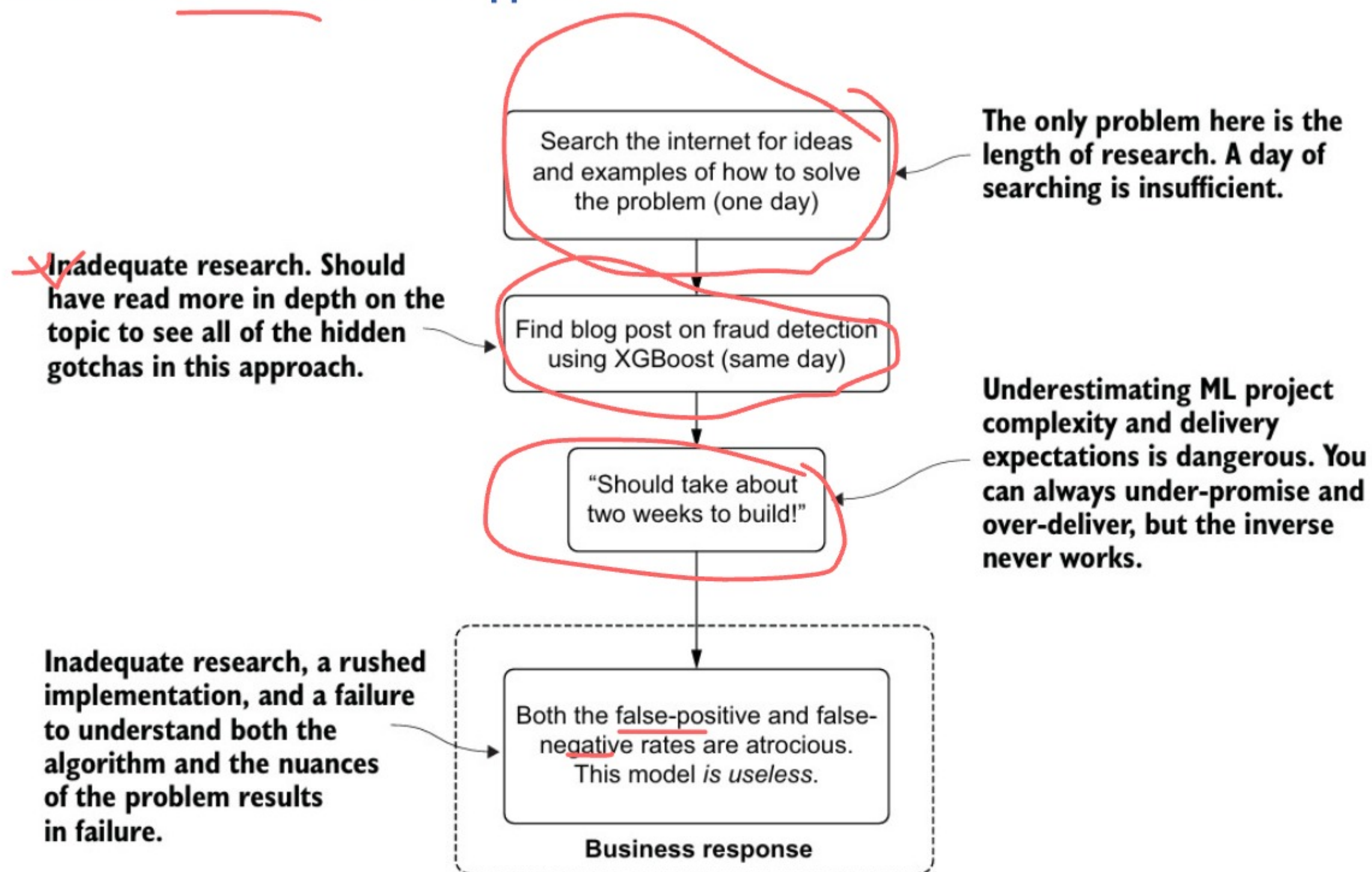
- 1) Will this solve my problem?
- 2) How long will it take?

But answering these isn't always straightforward, especially when different types of data science teams take very different approaches.



Vizuara AI Labs

Team A: The Fast but Shallow Approach

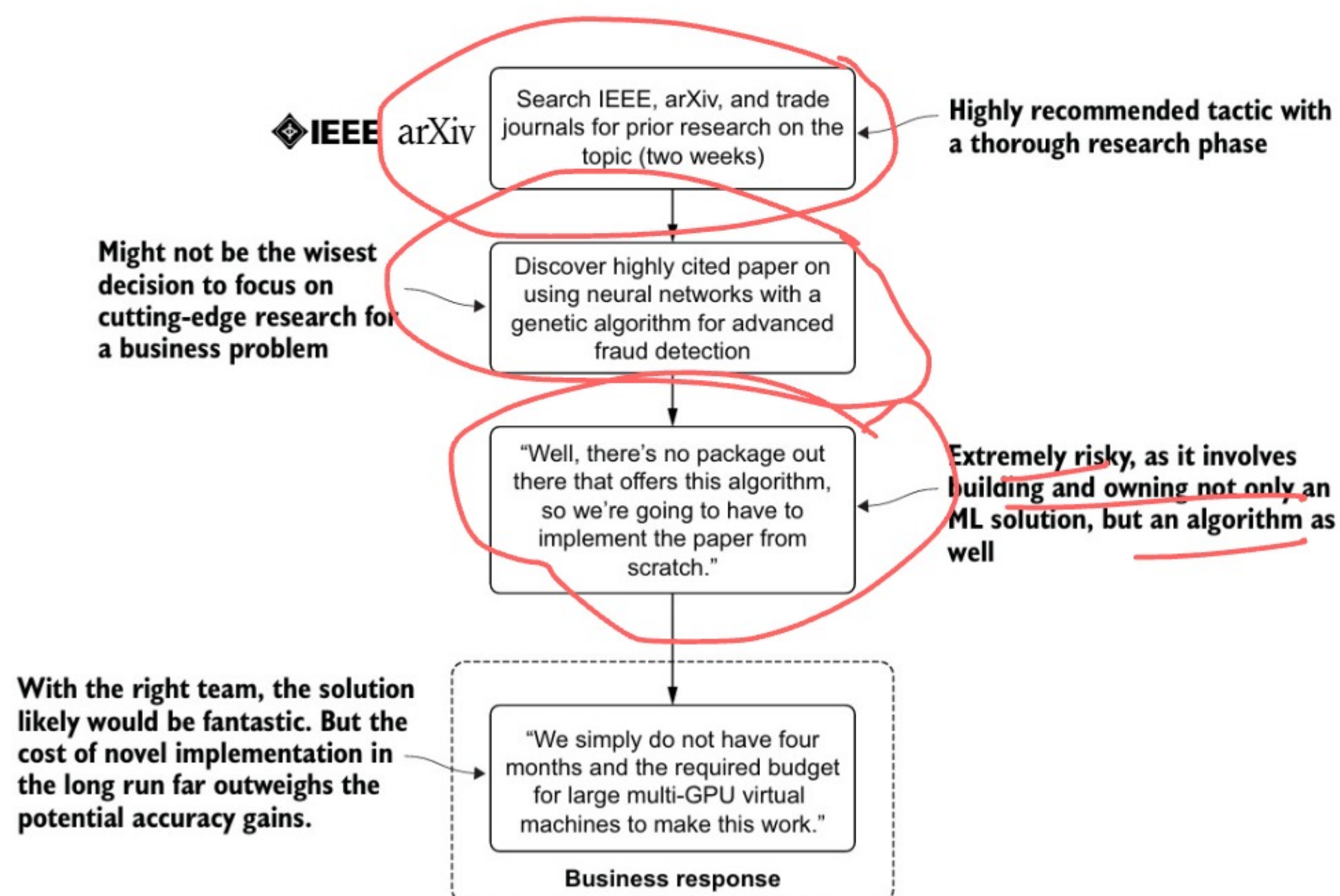


What went wrong? - The research was shallow only a day spent browsing beginner content.

- 1) They underestimated the complexity of fraud detection (e.g., class imbalance).
- 2) Their rushed approach led to poor results: a model with terrible false positive and false negative rates.

Outcome: The model didn't solve the problem. Time and effort were wasted.

Team B: The Deep but Risky Approach



What went wrong? - Too much time on theory and research, not enough on feasibility. Their approach was academically brilliant but business-unfriendly.
No existing package for their method meant they had to build from scratch.

Outcome: The idea was too costly and risky for business needs, so it was never implemented.

The Middle Path: Balanced Scoping Neither extreme works in real-world ML projects.

- 1) Too fast = under-researched, ineffective models.
- 2) Too slow = over-engineered, impractical solutions.

Best practice: Blend academic rigor with practical constraints. This means doing thoughtful but time-bounded research, testing feasible solutions early, and adjusting based on business context and resources.

2.3) Experimentation

Testing approaches is a Goldilocks activity; if you don't test enough options, you're probably not finding the best solution, while testing too many things wastes precious time. Find the middle ground.

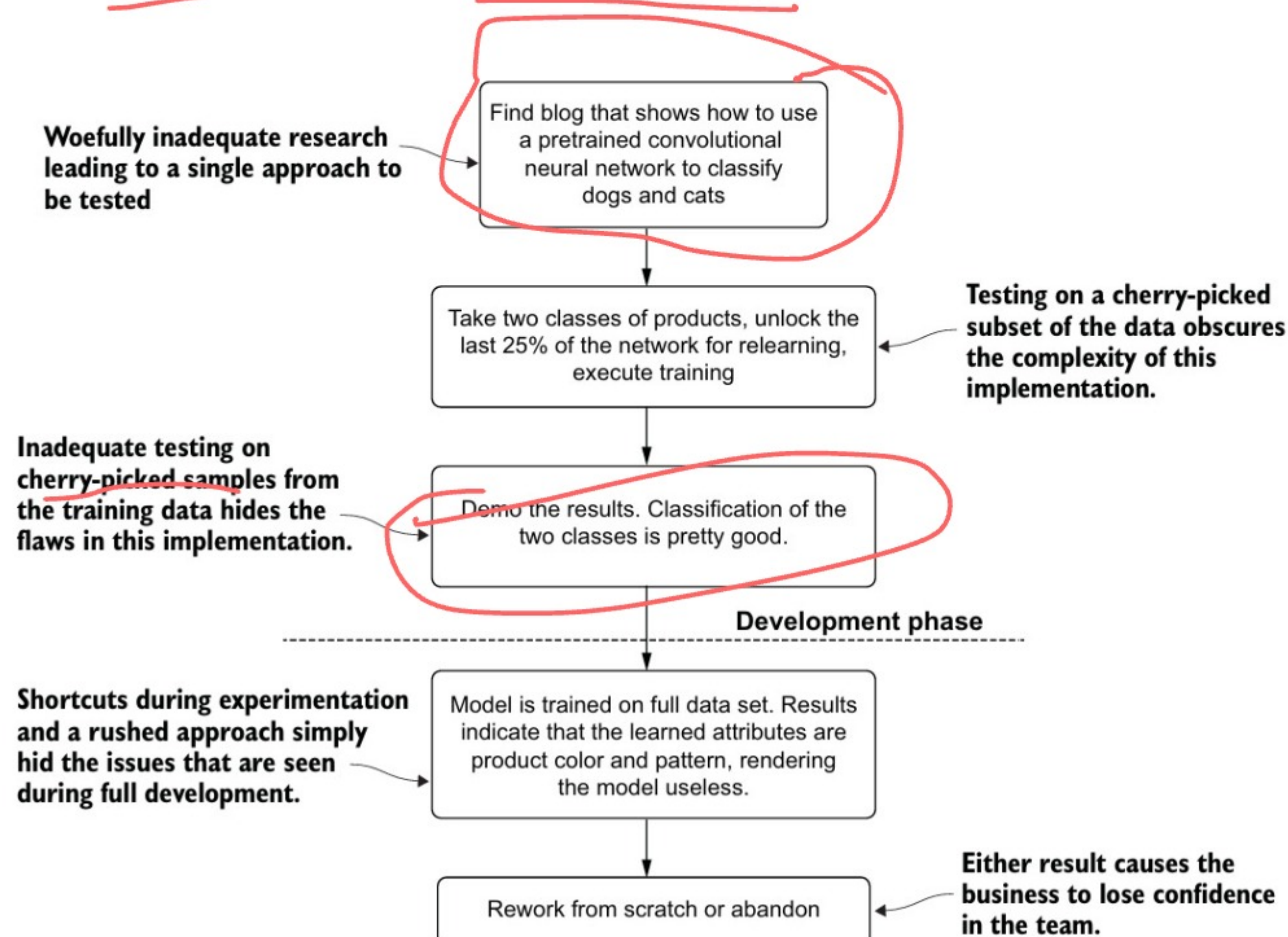
(Source:- Machine Learning Engineering in Action, Ben Wilson)

In the experimentation phase, the largest causes of project failure are:

- 1) Experimentation taking too long, often due to testing too many variations or over-tuning.
- 2) An underdeveloped prototype that performs so poorly it discourages stakeholders from continuing the project.

Let's explore this using an example of a company trying to build an image classifier to detect products on retail shelves. We'll contrast two extreme experimentation paths taken by two teams: Team A and Team B.

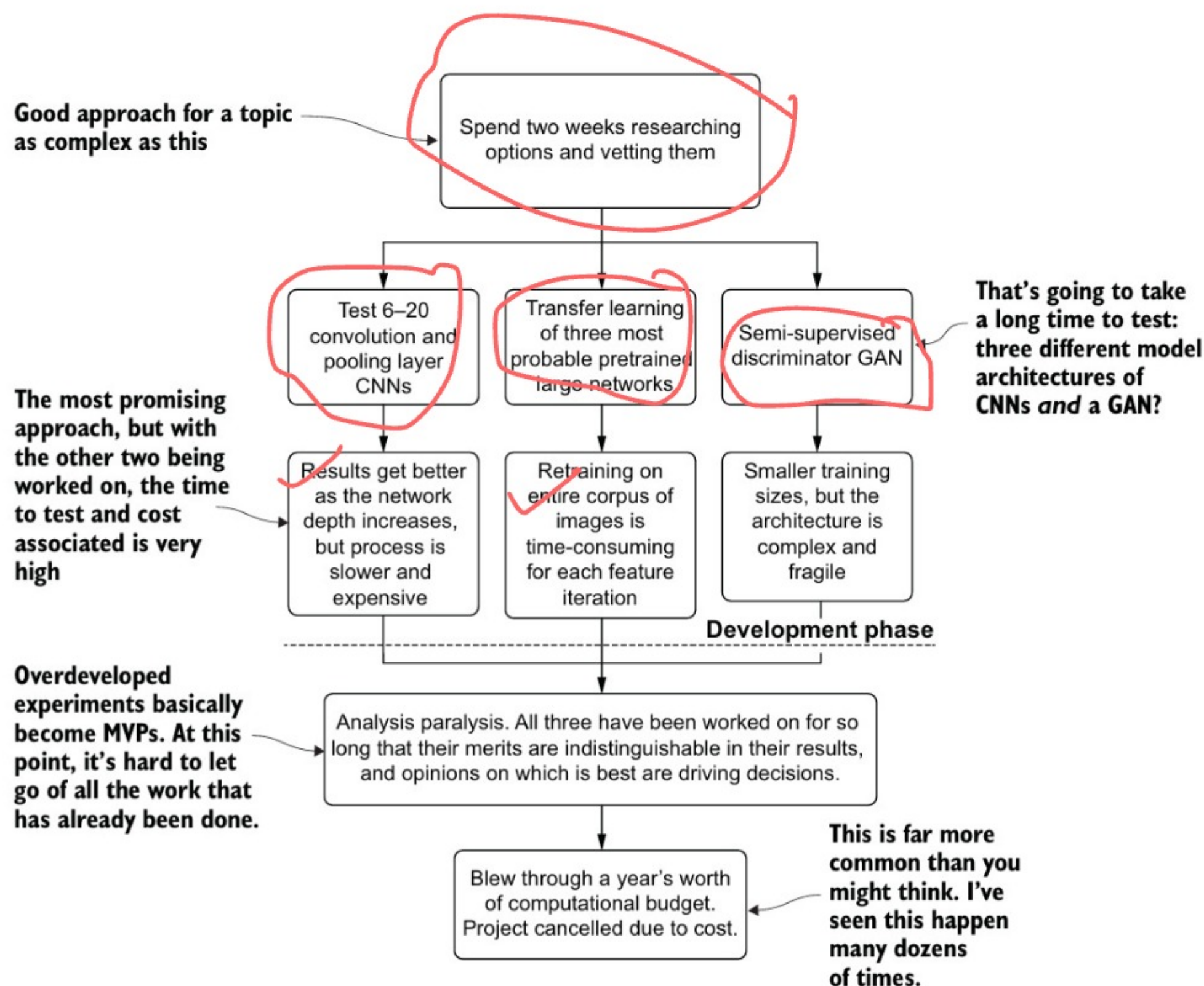
Team A: Rushed and Inadequate Experimentation



Outcome:

The project ultimately required either a complete rework or was abandoned altogether, resulting in wasted time, effort, and resources. This failure eroded the business's trust in the data science team and undermined confidence in their ability to deliver effective solutions.

Team B: Overly Ambitious and Exhaustive Experimentation



Outcome:

- 1) The project was cancelled due to uncontrolled experimentation that exhausted the entire computational budget without delivering a clear outcome.
- 2) Overdeveloped prototypes made it difficult to discard any single approach, leading to analysis paralysis.
- 3) Despite promising research, the inability to prioritize and make timely decisions caused stakeholders to lose confidence and pull the plug.

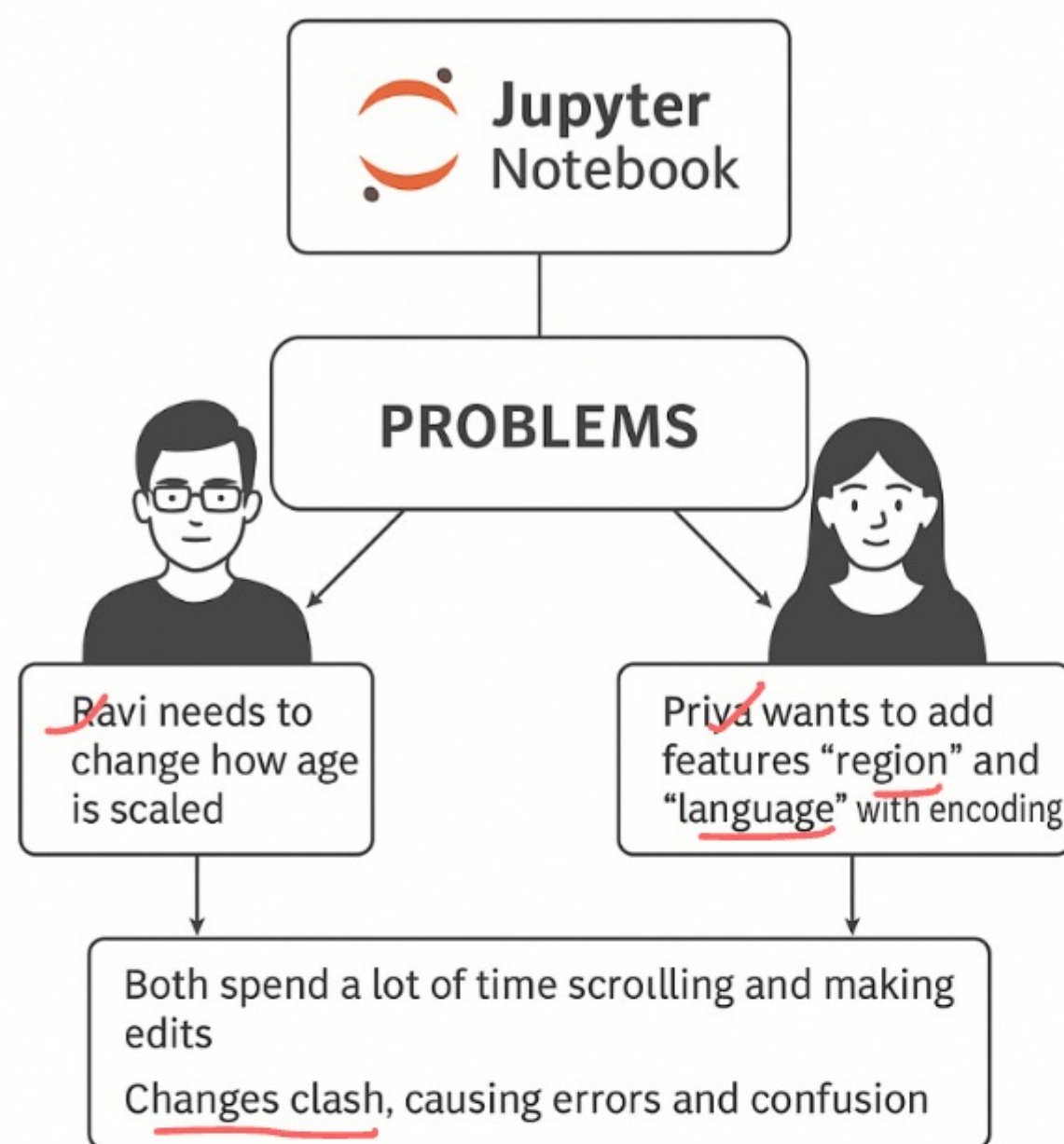
Both extremes, too little and too much experimentation, can derail projects.

A balanced, moderate approach is ideal:

- Test a few well-researched approaches.
- Move forward based on early results.
- Scope deep experimentation into the development stage, not the evaluation phase.

2.4) Development





Outcome:

Project progress slows down due to frequent merge conflicts and bugs. Time is wasted fixing basic integration issues instead of improving the model. The team loses confidence in the workflow and struggles to collaborate effectively.

Better Workflow (Modular Code):

Ravi and Priya use a modular codebase in VS Code, with separate files for feature creation, encoding, and scaling. Each checks out a feature branch from Git. Ravi updates the scaling module, Priya updates the encoding module, and both write simple tests.

Benefits:

They work independently without stepping on each other's changes. Their tests pass smoothly, and merging to the main branch is seamless after integration testing.



git



Outcome:

Changes are merged quickly and safely. Development becomes faster and more reliable. The team saves time and avoids rework, building trust in the process.

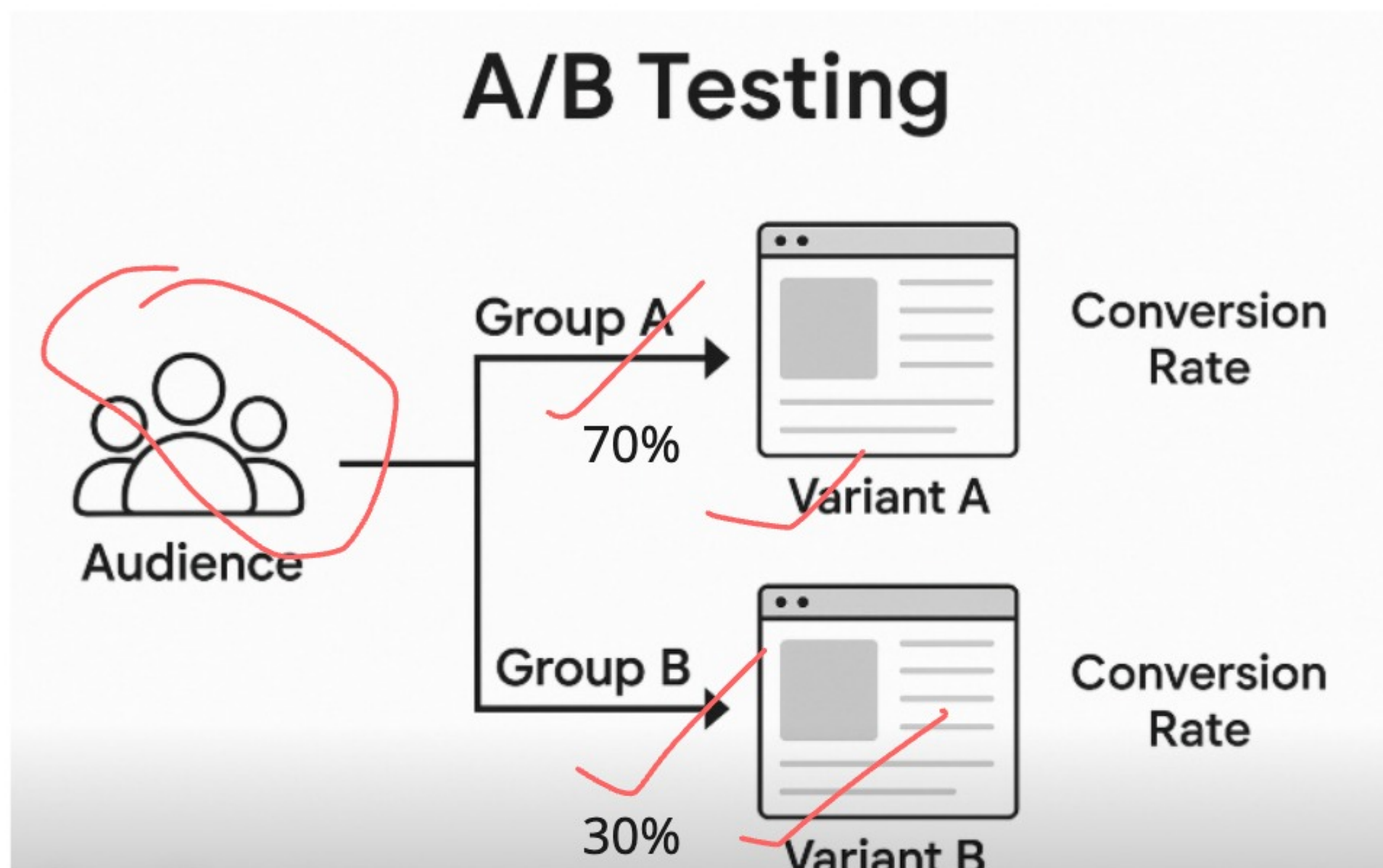


2.6) Evaluation

If you can't justify the benefits of your project being in production, don't expect it to remain there for very long.

(Source:- Machine Learning Engineering in Action, Ben Wilson)

- Even if a model performs well in production, if its business impact isn't clearly measurable, leadership might shut it down to cut costs. This often happens when teams fail to tie predictions to real revenue or ROI. Post-hoc validations aren't enough, attribution matters.
- Always plan for impact measurement early, use A/B testing and statistical models to estimate gains. Don't wait until executives ask, "What's the model really worth?" Have evidence ready.

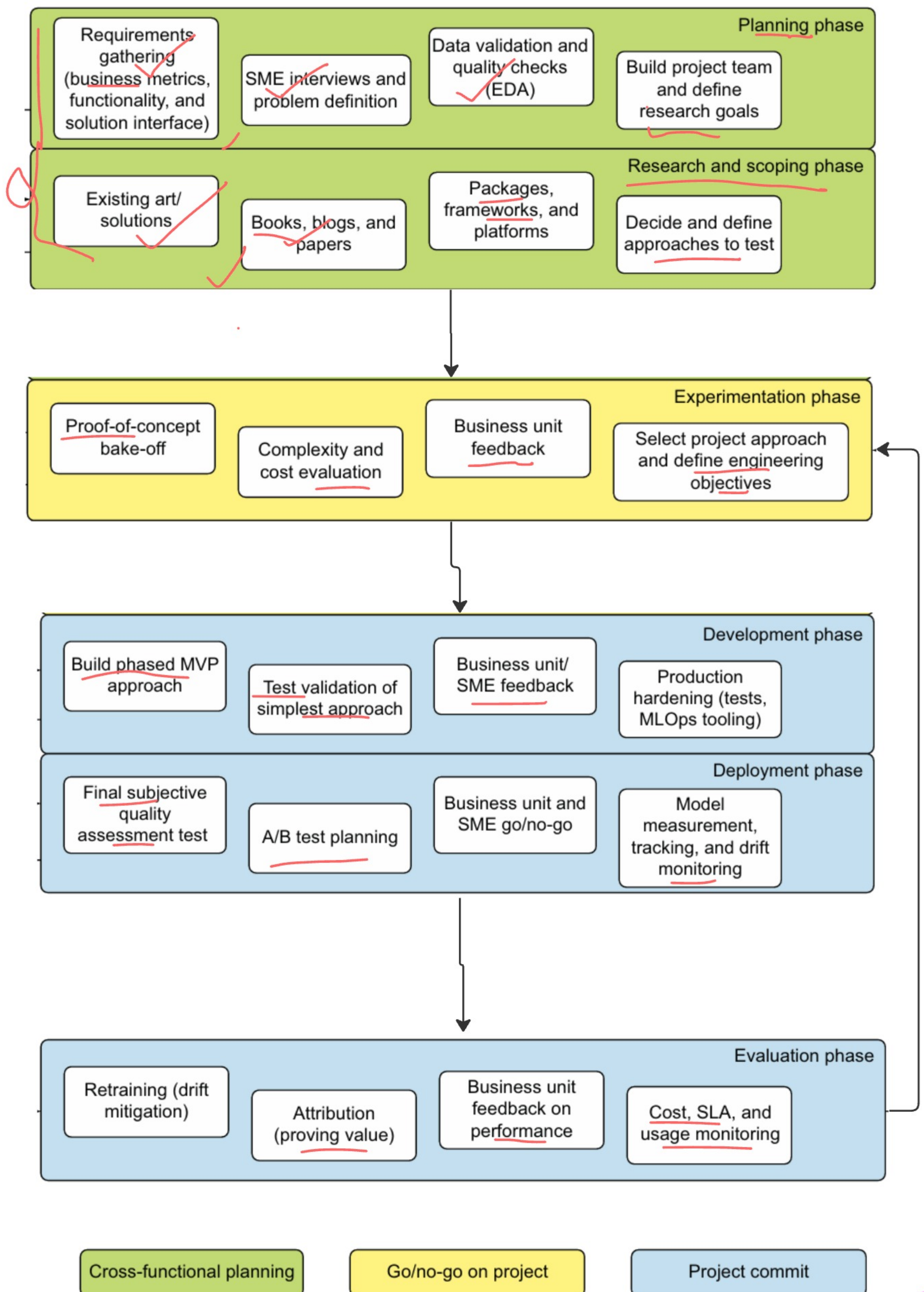


3) The goals of ML engineering

- 1) The core goal of a Data Scientist (DS) is to solve complex problems using statistics, algorithms, and predictive modeling.
- 2) The aim is not to build the most advanced model, publish flashy research, or use trendy technologies unnecessarily.
- 3) Data science exists to tackle tasks that are too complex, repetitive, or error-prone for humans.
- 4) DSs have access to a vast array of tools, which can sometimes lead to overemphasis on technical complexity.
- 5) Without a clear process guide, it's easy to lose focus on solving real-world problems.



4) Summarizing all these into a diagram



(Source:- Machine Learning Engineering in Action, Ben Wilson)

